

Deccan Education Society's
Fergusson College (Autonomous), Pune
Department of Computer Science

**A
Report
on**

“Step-by-step visualization of Bubble Sort and Insertion Sort behavior and efficiency, Real-time comparison of sorting time for various input conditions, Interactive input options: custom or predefined test cases.”

In partial fulfillment of Post Graduate course

In

M.Sc. Computer Science – I

(Semester -I)

CSC-520 Practical I

SUBMITTED BY

Arin Avinash Dhimar (264203)
Sunil Tarachand Rathod(246259)

Index

Sr. No	Table of Content	Page No
1	Case Study description (Algorithm details and UI design details)	3
2	Screenshots	5
3	References (if you have used any)	7

Algorithm Details

❖ Bubble Sort Algorithm

- Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.
- Time Complexity:
 - Best Case: $O(n^2)$
 - Average Case: $O(n^2)$
 - Worst Case: $O(n^2)$
- Space Complexity: $O(1)$

❖ Insertion Sort Algorithm

- Insertion Sort is an efficient sorting algorithm that builds the final sorted array one element at a time.
- Time Complexity:
 - Best Case: $O(n)$
 - Average Case: $O(n^2)$
 - Worst Case: $O(n^2)$
- Space Complexity: $O(1)$

‘UI Design Details

❖ Single Page Web Application

- The entire application is designed as a single-page web application. This allows for smooth transitions between sections and enhances the user experience by avoiding page reloads.

❖ Sticky Navbar

- The navigation bar is designed to be sticky, using the CSS position: sticky attribute. This ensures that the navbar remains visible at the top of the page as the user scrolls through the content. This feature provides easy access to different sections of the page without the need to scroll back up.

❖ Page Structure

- Introduction
- Bubble Sort
- Insertion Sort
- Comparison

❖ Section Height Management

- To create a clean and organized layout, each section (except the last) is given a height of 100vh (100% of the viewport height). This ensures that each section occupies the full screen. The last section, which contains substantial content, slightly deviates from this rule to accommodate the additional details.

❖ Bubble and Insertion Sort Sections Layout

- Left Column
- Right Column

The left column is further subdivided into three parts:

- **Introductory Section:** Explains the sorting algorithm.
- **Complexity Section:** Includes a simple jQuery toggle to show or hide the time complexity of the sorting algorithm.
- **Code Selection Section:** A dropdown (<select>) is provided for the user to choose from various programming languages. When the value of this dropdown changes, the code for the selected language is displayed using <pre> and <code> tags, preserving indentation for better readability.

❖ **Navbar Dynamic Highlighting**

- The navbar dynamically highlights the section that is currently being viewed. As the user scrolls through the page, the corresponding navigation link is highlighted, making it easy for users to know which section they are in.

❖ **Interactive Input Options**

- The application provides a dropdown (`<select>`) tag that contains predefined input options. Users can select an option to generate random numbers using `Math.random()`, which are then added to the array for visualization. This dynamic input generation allows users to quickly visualize sorting for various scenarios.

❖ **Array Display Management**

- The input values are added to the array-display block, which uses the `display: flex` property. The items inside this container wrap to the next line if necessary, and a gap of 5px is maintained between them. The items' width is dynamically calculated using `calc(1% - 10px)`, which ensures that each item takes up 1% of the container width minus the gap, making the layout responsive and adjustable.

❖ **Dynamic Array Boxes Creation**

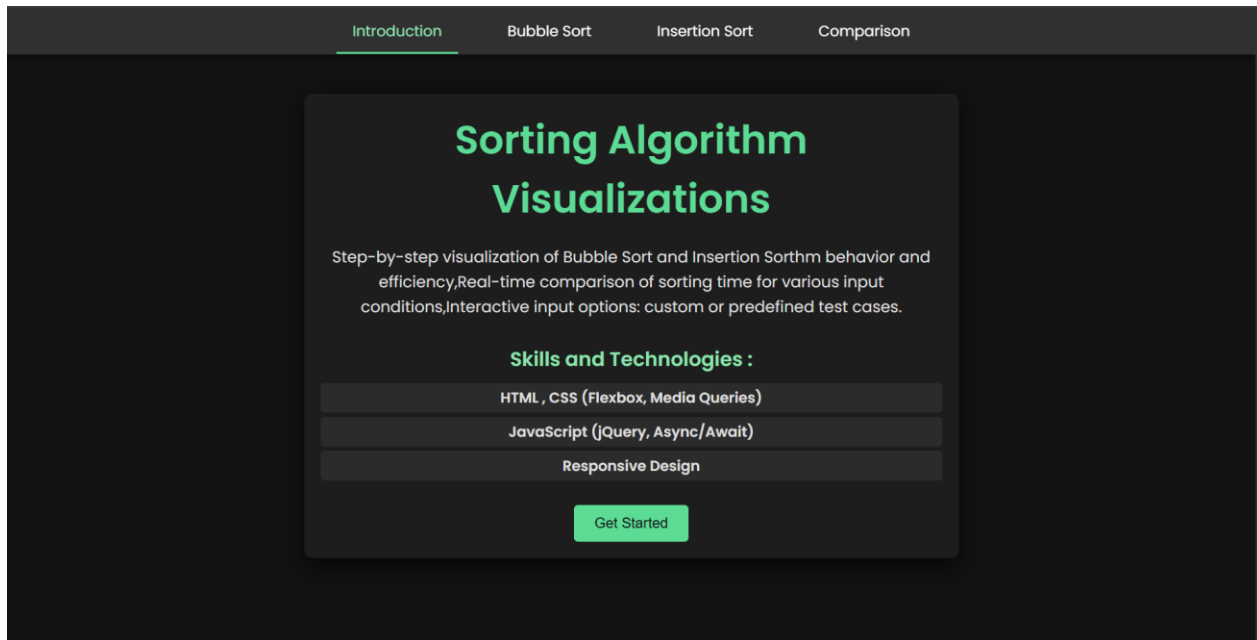
- Each time a new value is added to the array, a new div with the `array-box` class is created. The value of the item is set as the text of the div, and it is appended to the `#arrayDisplay` container.

❖ **Visualization of Sorting Process**

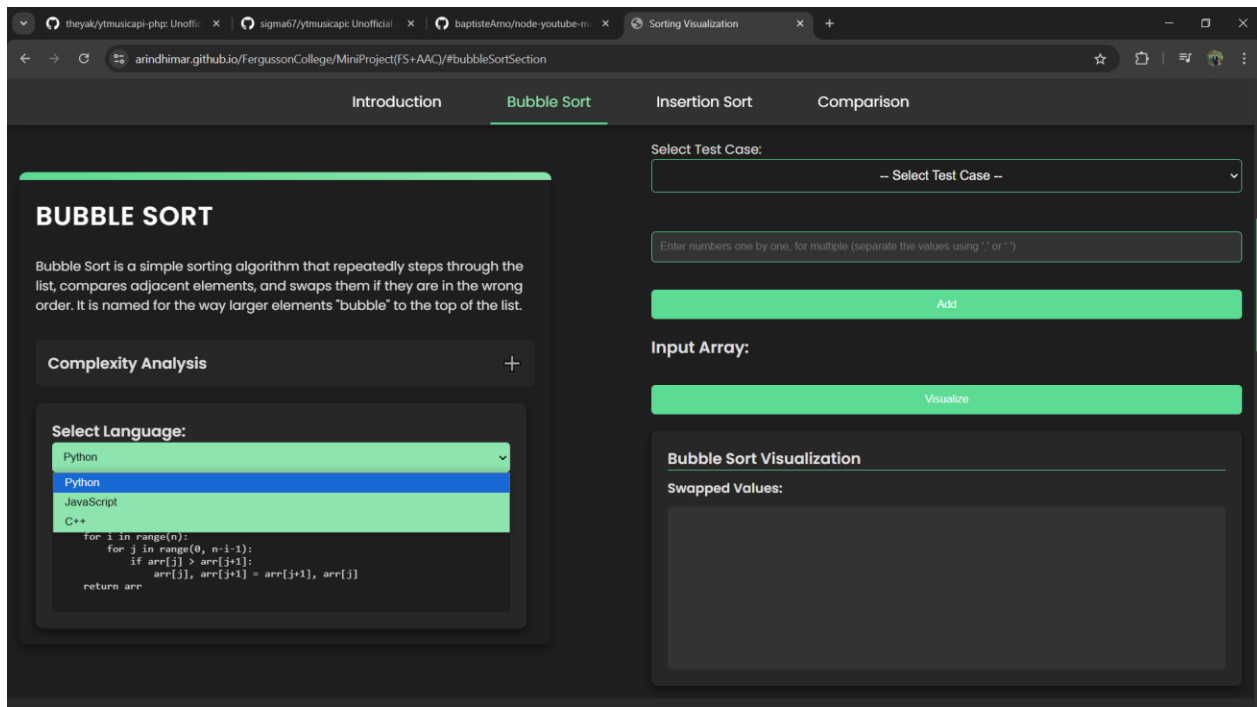
- **Array Box Removal:** All the array-box elements are removed from the display.
- **Loop Through the Array:** The array is iterated through, and for each element, one box is highlighted in red (indicating the element being swapped) and the other is highlighted in yellow (indicating the element being compared).
- **Promise, Async, Await:** The sorting process uses Promises and `async/await` to handle the animation and delay. This ensures smooth transitions between comparisons and swaps, and provides a brief pause to visually represent each step of the sorting process.

Screenshots:

- **Introduction Page:**



- **Bubble Sort:**



- **Insertion Sort:**

Introduction

Bubble Sort

Insertion Sort

Comparison

INSERTION SORT

Insertion Sort is an efficient sorting algorithm that builds the final sorted array one item at a time. It is much less efficient on large lists than more advanced algorithms like quicksort or merge sort.

Complexity Analysis

Select Language:

Python

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
```

Select Test Case:

Average Case

-- Select Test Case --

Best Case

Average Case

Worst Case

Add

Input Array:

89

7

40

35

73

25

77

59

82

98

Visualize

Insertion Sort Visualization

Current State:

- **Comparison:**

Introduction

Bubble Sort

Insertion Sort

Comparison

16

39

89

1

3

74

76

13

26

3

Bubble Sort

Bubble Sort Visualization

Intermediate Values:

Comparing: 89 and 13

16

39

1

3

74

76

89

13

26

3

Time Taken: 5.12s

Total Swaps: 4

Insertion Sort

Insertion Sort Visualization

Intermediate Values:

Key: 3, Comparing with: 39

1

16

39

89

89

74

76

13

26

3

Time Taken: 4.61s

Total Swaps: 3